

Language Models

From Words to Numbers

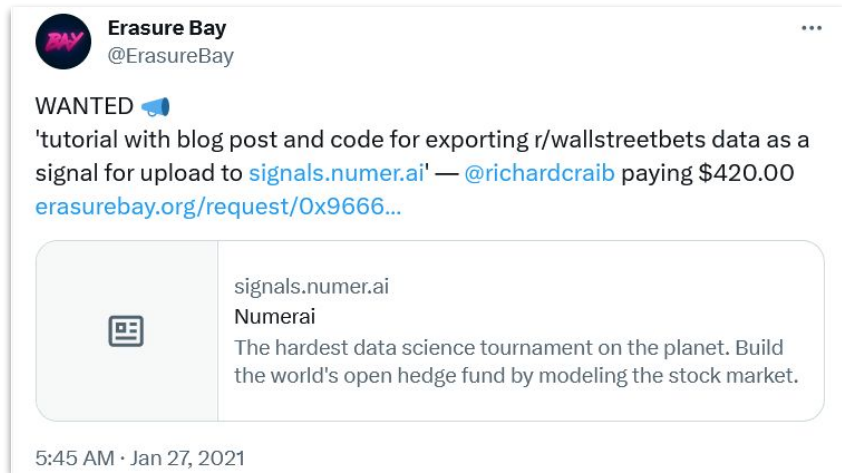
parmarsuraj.com

Discord: @surajp1729

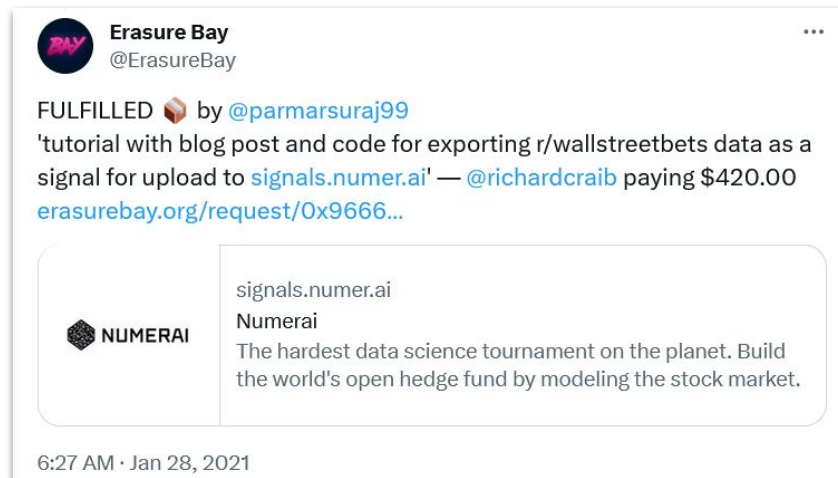
Slides: <https://tinyurl.com/lm-toronto>

Code: within slides

Why?



Long on \$Stock
Diamond hands, YOLO
\$Stock to the moon!
Quarter earnings bad



From r/WSB to Numerai Signals



Suraj Parmar

3 min read · Jan 28, 2021

[Lex Fridman finds it interesting! #159](#)

Language Modelling is [...]

- At its core, language modeling is about predicting the next word in a sequence of words.
- For instance, in the sentence, “Hello there, how are you doing? Today is a beautiful day. It’s Saturday. So, it’s a _____”
 - Sunday
 - Holiday
 - Water
 - Canada

Language Modelling

- Statistical approaches
 - N-gram models
- Neural approaches
 - MLP
 - LSTM
 - Transformers

N-grams: Bringing Math to Words

- From rule based to probability
- “Contagious sequence of ‘N’ items from given sample of text”
- From a given text, we calculate all ‘N’ contagious items or words.
[W1, W2, W3, W1, W3, W6, ..., Wd]
- We calculate probability of all unique sequences appearing together.
- What is the probability of W1, W2, W3 appearing in a sequence?
 $P(W1, W2, W3)$?

N-grams: examples

- Corpus/book: “I love language models. I am talking about them today.”
 - Uni-gram: ['I', 'love', 'language', ...]
 - Bi-gram: ['I love', 'love language', ... 'I am', 'them today']
 - Trigram: ['I love language', 'love language models', ...]
- $P(\text{“I love Language models”})$ using bigram:
$$\sim P(I) * P(\text{love}|I) * P(\text{language}|\text{love}) * P(\text{models}|\text{language})$$
- Probability of an unforeseen sequence? $P(\text{'I love languages and computer'})$
 - Smoothing for OOD (add 1 and divide by V for 0 probabilities)
 - backoff (switch to lower n-gram)

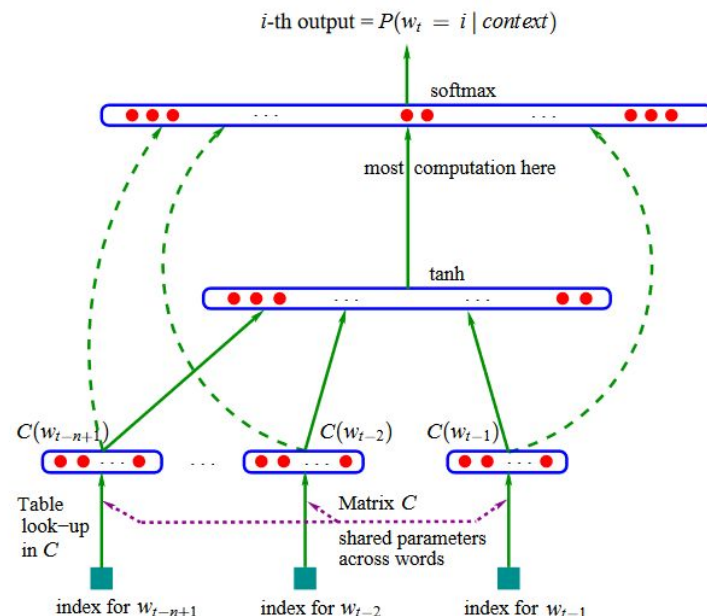
N-grams: Challenges

- Challenges:
 - Sparsity – probability of most combinations is 0.
 - Fixed window size
 - Overfit on given corpus – almost no generalization
 - Storage – 3-gram model, 2000 words
 - No semantic meaning
- Simple applications:
 - Next word suggestion on phone
 - Spell correction

Deep Learning Approaches

- “A Neural Probabilistic Language Model”

- **Embeddings Over Counts:** Words become vectors, capturing semantic richness.
- One-hot (17,000) -> embeddings (100)
- Contextually aware word vectors
- **Why Neural?:** Beyond counting – a deeper, learned understanding of language.
- **Origins:** A major shift in NLP introduced by Bengio et al. in 2001.



“A Neural Probabilistic Language Model”

- **One-hot Encoding:**

- For {cat, mat, roof, apple}, "cat" is [1, 0, 0, 0].
- Unique binary representation for each word in the vocabulary.

- **Input:**

- Words are transformed into a compact, dense space where semantic relationships are captured.
- From one-hot encoding of 17,000 dimensions size to dense vector of 100 dimensions.
- "king" in the neural space is closer to "queen" than "apple".

- **Hidden Layers:**

- Network processes context and relationships between words to make predictions.
- For "The cat is on the _____," it leans towards "mat" or "roof".

- **Output:**

- After "The cat is on the", it gives higher chances for "mat" and "roof" over "apple".
- Probabilistic predictions for the next word based on the given context using SoftMax.

Words-to-Numbers, Refined. Word2Vec

- From NPLM:

- Introduced word embeddings: Transformed words into numerical vectors.
- Goal: Understand word context and represent semantically.

- Challenges of NPLM:

- Computational Intensity: Required significant resources, especially with large vocabularies.
- Nuance & Efficiency: While innovative, capturing intricate word relationships remained a challenge.

- To Word2Vec:

- Optimization: More efficient embeddings with fewer resources.
- Architectures: Introduced two primary methods - Skip-gram (predicting context from a word) & CBOW (predicting a word from its context).
- Improved Semantics: Better captured relationships and nuances between words.

Word2Vec - intuition

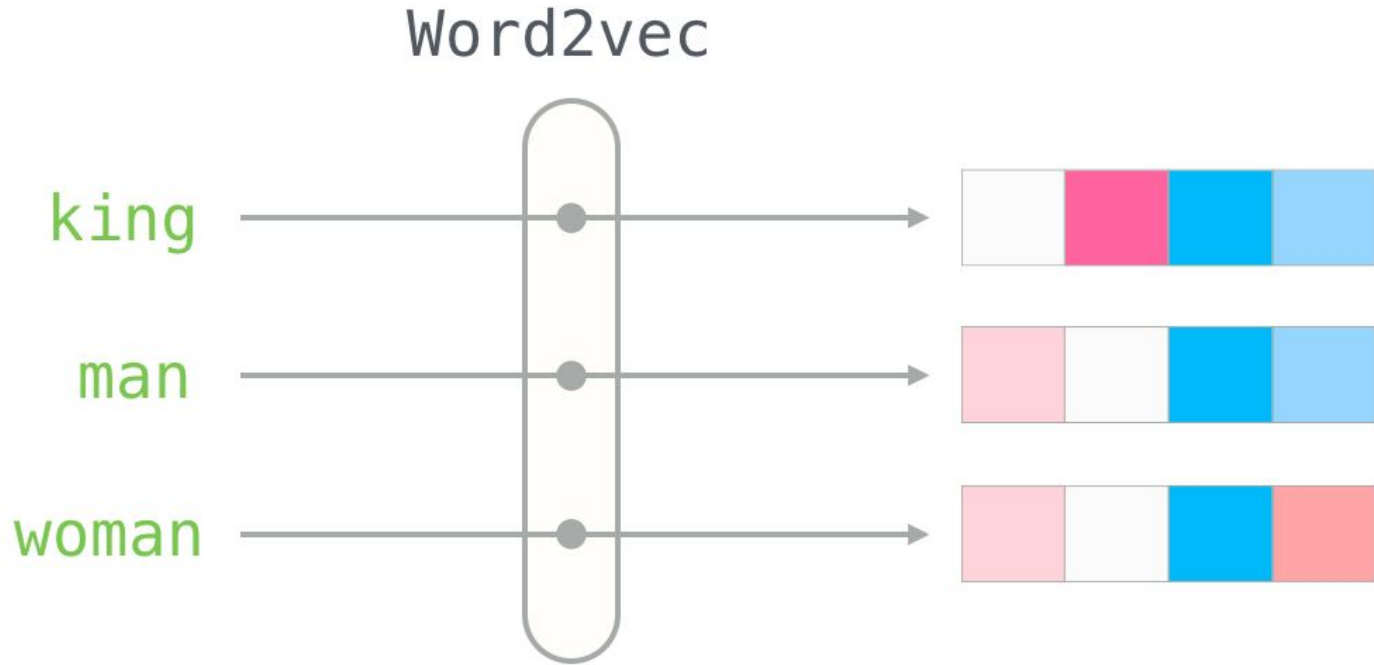


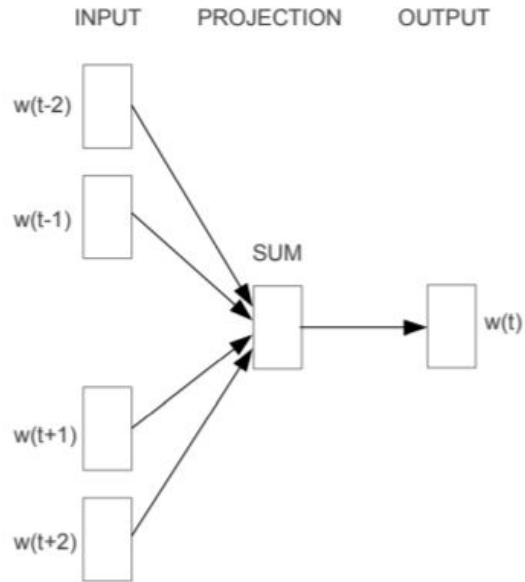
Image:

<http://jalammar.github.io/illustrated-word2vec/>

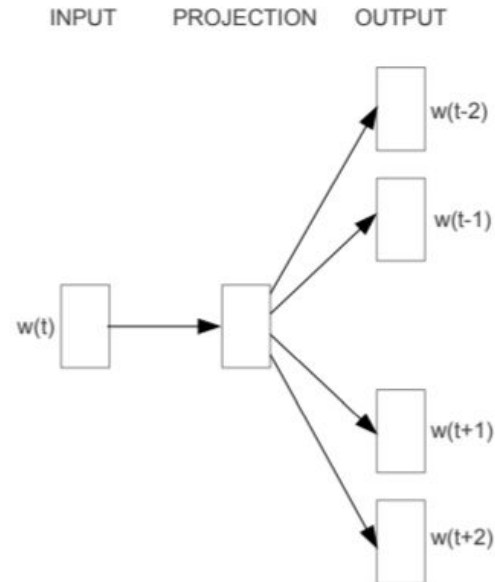
Word2Vec – overview

- Two Primary Architectures:
 - CBOW (Continuous Bag-of-Words): Predicts a target word using its surrounding context.
 - Skip-gram: The inverse - predicts context words from a target word.
- Key Components:
 - **Embedding Matrix:** Represents each word as a dense vector.
 - **Context Window:** Defines the number of words considered as context.
 - **Negative Sampling:** Improves efficiency by sampling negative examples.
- Training Process:
 - Adjust word vectors to minimize prediction errors.
 - Uses gradient descent.

Word2Vec - training



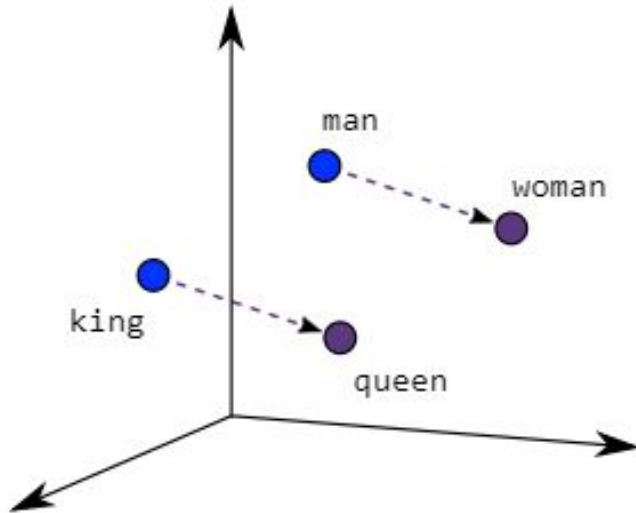
CBOW



Skip-gram

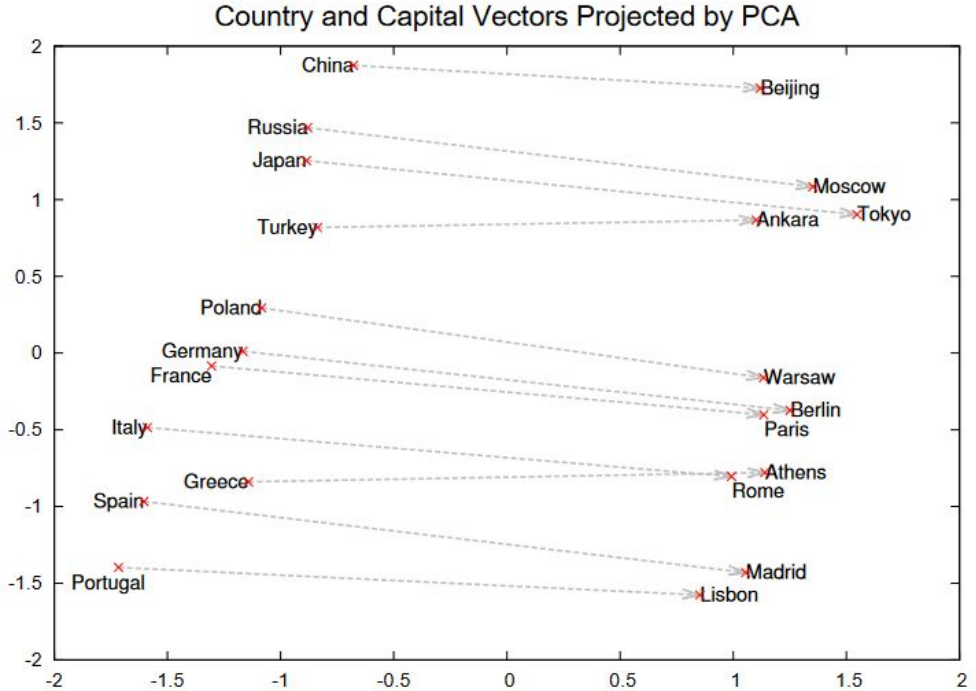
“Efficient estimation of word representations in vector space”
Mikolov et.al.

Word2Vec – interesting!



$\text{king} - \text{man} + \text{woman} = \text{queen}$

[source](#)



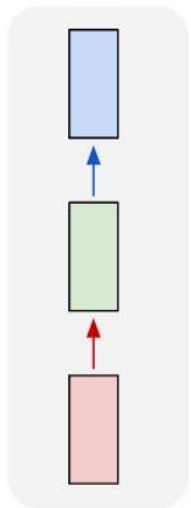
“Distributed Representations of Words and Phrases and their Compositionality” Mikolov et.al.

Word2Vec – Applications

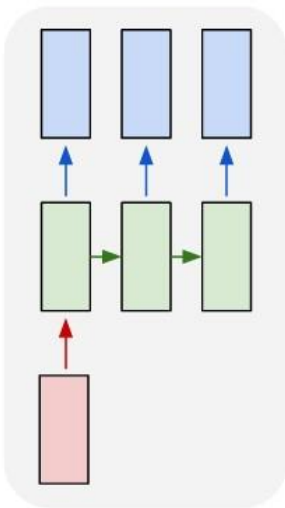
- Semantic Understanding:
 - Word similarities and analogies (e.g., Man : Woman :: King : Queen).
 - Enables machines to grasp word meanings.
- Natural Language Processing Tasks:
 - Sentiment Analysis: Understand sentiments in text.
 - Text Classification: Categorizing documents or articles.
- Information Retrieval:
 - Enhancing search engines.
 - Recommender systems based on textual content.
- Other Domains:
 - Biology: Gene function prediction from literature.
 - Finance: Analyzing sentiments in financial reports.

Processing Sequences

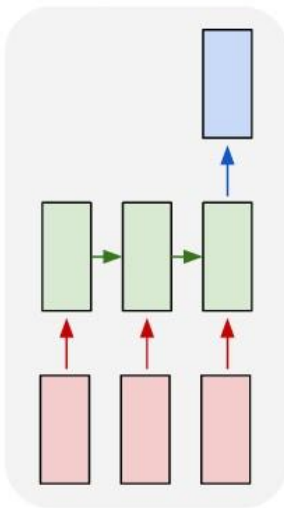
one to one



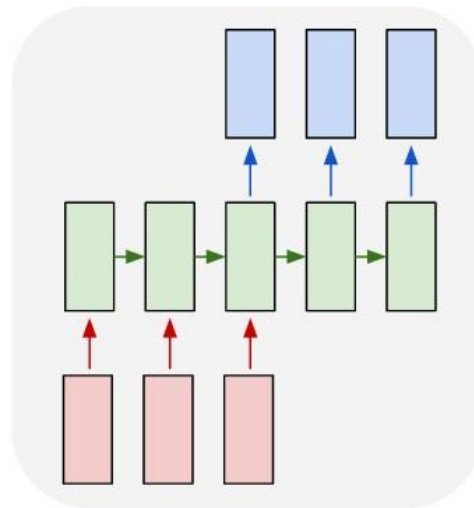
one to many



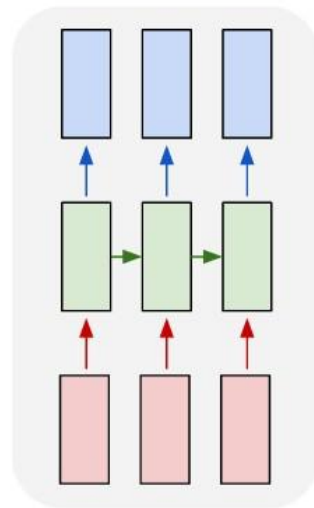
many to one



many to many



many to many



<http://karpathy.github.io/2015/05/21/rnn-effectiveness/>

Processing Sequences – The LSTM Leap

- **Evolution from RNNs:**

- RNNs: Captured sequences but had memory issues.
- LSTM: A specialized RNN variant addressing those limitations.

- **Key Strengths:**

- Long-term dependencies captured.
- Solves vanishing & exploding gradient problems.

- **Applications:**

- Text generation, machine translation, and more.

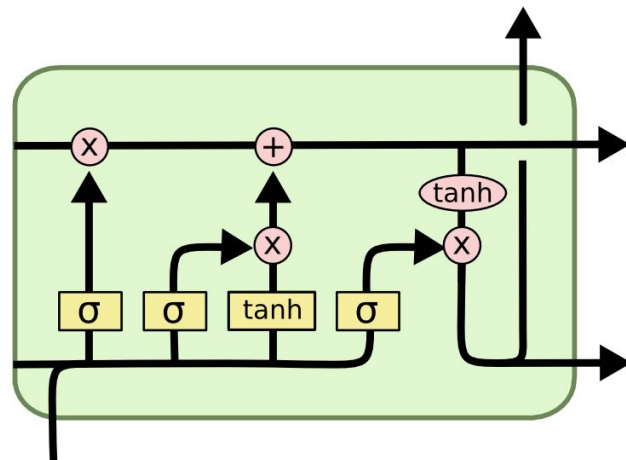
Processing Sequences – Inside LSTM

- **Gates & Memory:**

- Input, forget, and output gates.
- Maintains, modifies, and uses memory effectively.

- **Sequential Learning:**

- Understands time-based patterns and structures.
- Retains crucial information across long sequences.



<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Transfer Learning in NLP - ULMFiT

- **Universal Language Model Fine-tuning:**

- Transfer learning approach for NLP tasks.

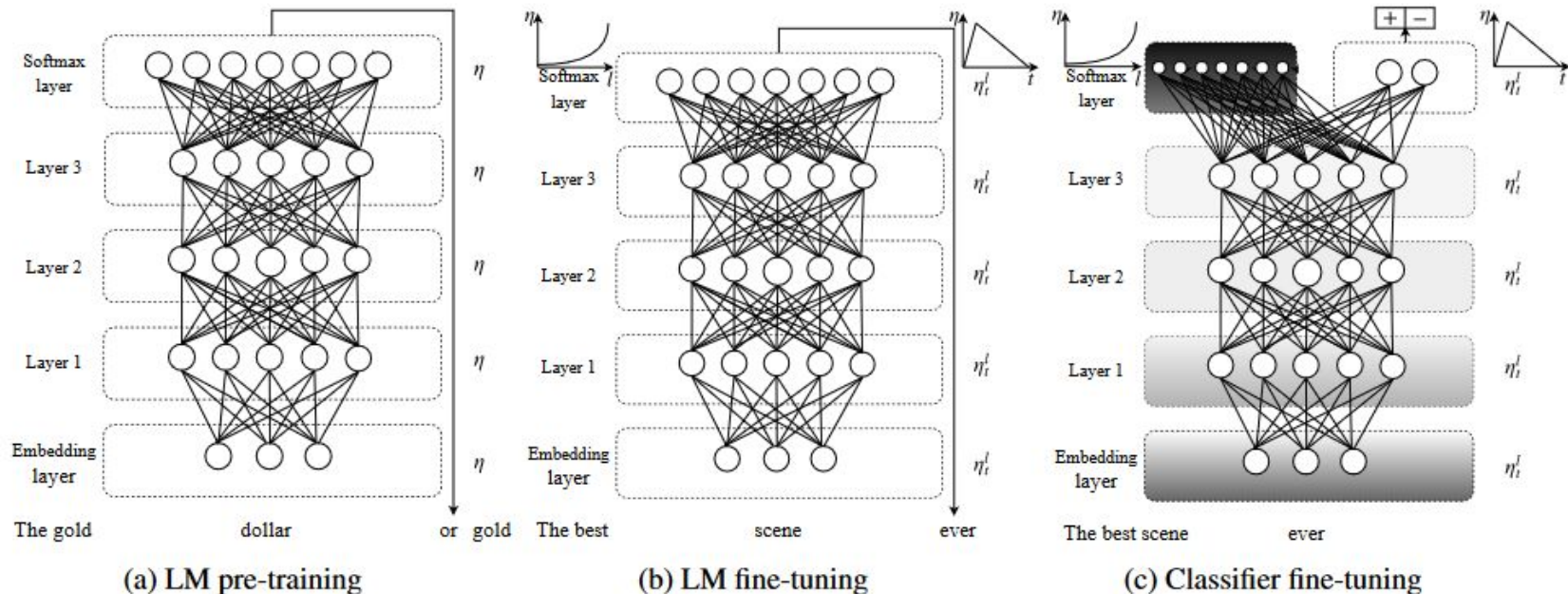
- **Stages:**

- Pretrain on general domain.
- Fine-tune on specific task.

- **Advantages:**

- Faster convergence.
- Lower data requirements for high performance.

Transfer Learning in NLP - ULMFiT



[Universal Language Model Fine-tuning for Text Classification](#)

(Howard & Ruder, ACL 2018)

Why ULMFiT Matters

- **Breaking Barriers:**

- Demonstrated transfer learning's potential in NLP.
- Revolutionized task-specific model training.

- **Fine-tuning Nuances:**

- Discriminative fine-tuning.
- Gradual unfreezing.

natural language processing

NLP's ImageNet moment has arrived

Big changes are underway in the world of NLP. The long reign of word vectors as NLP's core representation technique has seen an exciting new line of challengers emerge. These approaches demonstrated that pretrained language models can achieve state-of-the-art results and herald a watershed moment.



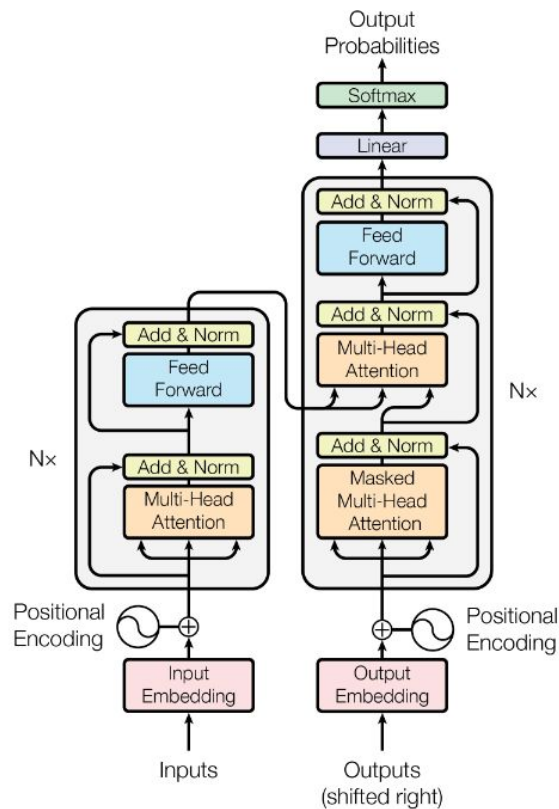
Sebastian Ruder

Jul 12, 2018 • 16 min read

<https://www.ruder.io/nlp-imagenet/>

“Attention Is All You Need” - Transformers

- **Previous Model Limitations:**
 - RNNs, LSTMs: Struggled with long-range dependencies.
 - Scalability and parallel processing challenges.
- **Transformers' Core Strength: Attention Mechanism:**
 - "Attention" identifies which parts of the text are relevant.
 - Allows for simultaneous processing of words/phrases.
- **Paradigm Shift:**
 - From sequential to parallel processing in NLP.
 - Enabled groundbreaking models like BERT & GPT.



Attention Is All You Need Paper: <https://arxiv.org/abs/1706.03762>

Tokenization: The Foundation of Transformers

- **The Need for Tokenization:**

- **Definition:** Breaking down text into smaller pieces, often words or subwords.
- Enables models to process and understand text data.

- **Subword Tokenization:**

- Common words remain whole, while rare words are split into smaller pieces.
- E.g., "transformer" might be tokenized as ["trans", "form", "er"].

- **Benefits:**

- Handles a vast range of vocabulary, including unseen words.
- Provides a balance between word-level and character-level tokenization.

- **Real-World Example:**

- Sentence: "ChatGPT assists efficiently."
- Tokenized: ["Chat", "G", "PT", " assists", " efficiently"]

BERT

- **Bidirectional Understanding:**

- Previous models read text one-way (left to right or right to left).
- BERT processes both directions, ensuring deeper context.

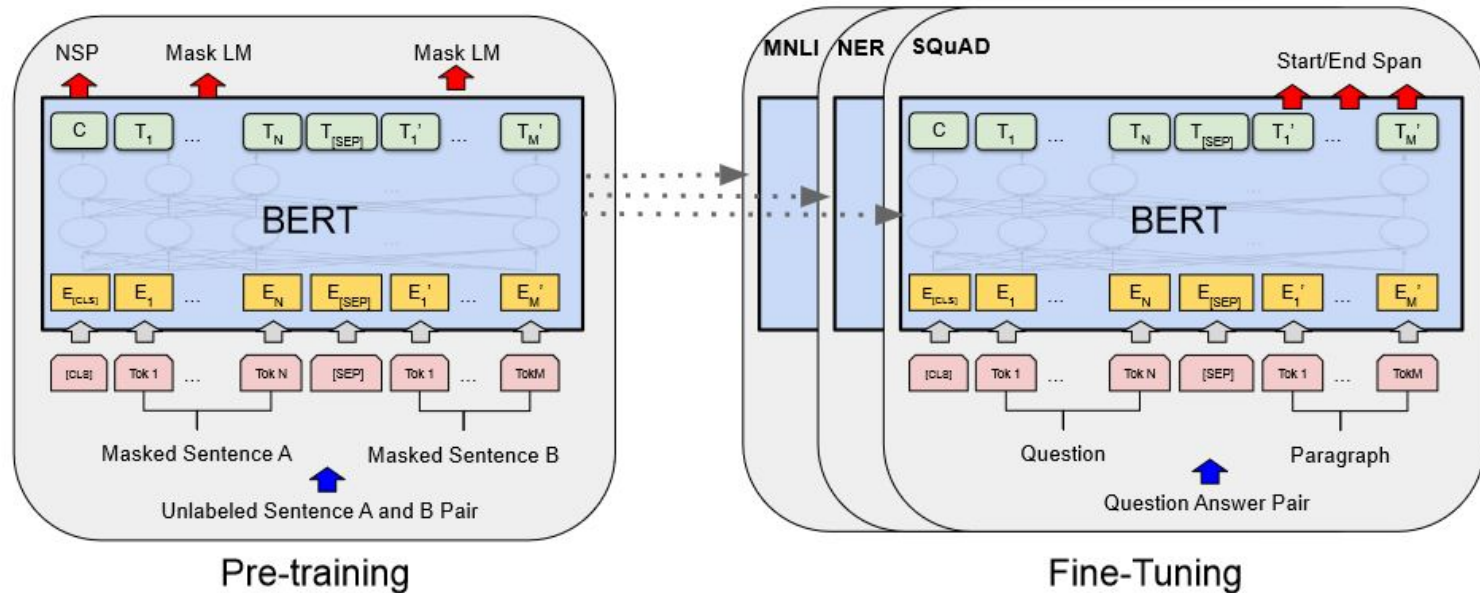
- **Training with a Twist: Masked Language Model:**

- Random words in a text are hidden ("masked") during training.
- Trained to predict these masked words from surrounding context.

- **Fine-tuning Flexibility:**

- Pre-trained on large datasets and fine-tuned on specific tasks.
- Versatile across numerous NLP challenges.

BERT



BERT Paper: <https://arxiv.org/abs/1810.04805>

BERT's NLP Domination

- **Dynamic Word Representations:**
 - Word meanings vary based on context. E.g., "bank" in "river bank" vs. "savings bank".
 - BERT captures these nuances effectively.
- **State-of-the-Art Benchmarks:**
 - Outperformed predecessors in 11 of 11 NLP tasks upon debut.
 - Led to a surge in transformer-based models.
- **Real-World Impact:**
 - Search engines, chatbots, and content recommendation enhancements.

GPT: The Generative Powerhouse

- **What Sets GPT Apart:**

- Decoder Model
- While BERT excels at **understanding** text, GPT is designed to **generate** it.

- **Training Paradigm:**

- Uses a left-to-right transformer. Predicts the next word in a sequence.
- Trained on diverse internet text but doesn't know specifics.

- **Scalability Advantage:**

- GPT-2 (1.5B parameters), GPT-3 (175B parameters): Demonstrates the power of scaling.

GPT: Changing the Landscape

- **Human-like Text Generation:**

- Can write essays, poems, answer questions, and more.

- **Few-shot Learning Ability:**

- Given a few examples, GPT can perform tasks it was never explicitly trained on.

- **Broad Application Spectrum:**

- Gaming, coding assistance, content creation, tutoring, and beyond.

Customizing Large (generative) LM

1. Unsupervised Pre-training

1. Take raw corpus (e.g. whole internet)
2. Model learns to predict next token

2. Supervised fine-tuning

1. Relatively smaller text for specific purpose
2. Ask it to complete instructions or
3. Train it to continue conversations

My early experiments - 2020

- **Goal:** To be able to ask a question in Hindi and retrieve the document in Sanskrit.
1. Not many Hindi models on HF Hub. Let alone Sanskrit.
 2. Collected data for Hindi, Gujarati and Sanskrit
 3. Trained multiple Encoder models and fine-tuned existing Hindi models on combination of languages
 4. Built retrieval for Sanskrit texts
 1. Transform Sanskrit sentences into vectors,
 2. Transform Hindi query into vector,
 3. Find most similar ones.
 5. Also trained a GPT-2 based Hindi model from scratch

 surajp/gpt2-hindi

 Text Generation • Updated Jun 21 • ⬇ 80

 surajp/albert-base-sanskrit

 Feature Extraction • Updated Jun 21 • ⬇ 8 • ❤ 3

 surajp/roBERTa-hindi-guj-san

 Fill-Mask • Updated Jun 21 • ❤ 1

 surajp/SanBERTa

 Fill-Mask • Updated Aug 11

 surajp/sanbert-from-indicbert

 Fill-Mask • Updated Apr 18, 2022

 surajp/sanbert-from-scratch

 Fill-Mask • Updated Apr 18, 2022 • ❤ 1

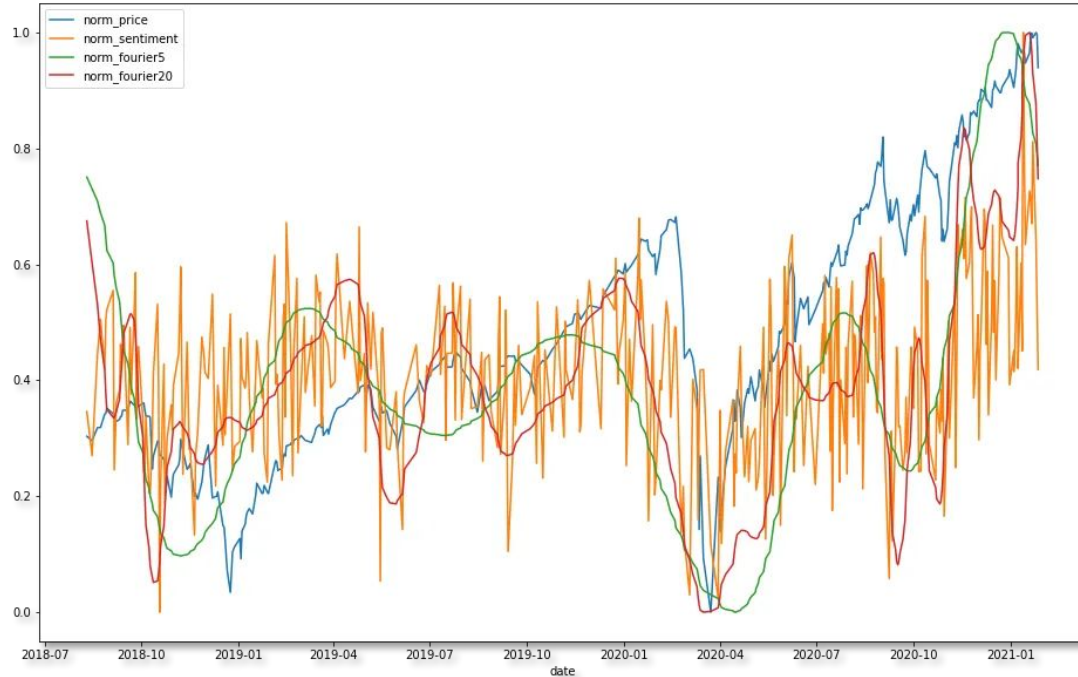
PunctFormer – Punctuation restoration 2020

1. **P**unctuation restoration as translation with Trans**formers**.
2. Bart (seq2seq) model to restore punctuation
3. Helps in parsing text from Automatic Speech Recognition (ASR)



<https://medium.com/da-labs/punctformer-bba0c5485d81>

Sentiment Model x r/WSB



From r/WSB to Numerai Signals



Suraj Parmar

3 min read · Jan 28, 2021

<https://pamarsuraj99.medium.com/from-r-wsb-to-signals-684194364022>

> Has Colab as well.

ChatGPT x r/WSB

🔍 ['No ticker:0:No ticker, VIX:0:No sentiment, BLK:-0.5:Bearish, TQQQ:-0.5:Bearish, UNH:0.5:Bullish, JPM:-0.5:Bearish, SPY:0.5:Bullish, SQ
['FXHedge:0.0:FXHedge, NBC:0.0:NBC']
['NVDA:0.7:bulls, frantic, typing, AI, RED, gap, AAPL:-0.7:ATH, declining, earnings, F:0.2:truck, tranny, Bud, Light, billion, SPY:0.8:bullish, recessi
['AMZN:-0.2:short-term headwinds, AAPL:0.8:25k profit, MSFT:0.7:birthday, NVDA:0.7:AI cloud, S&P:0.7:pump, GOLD:0.7:cooler inflation, QQ
['AAPL:0.0:neutral, NFLX:0.0:neutral, NVDA:0.0:neutral, WFC:0.0:neutral, JPM:0.0:neutral, SPY:0.0:neutral, META:0.0:neutral, F:0.0:negative, AMD
['NVDA:+1:buying, JPOW:+1:accomplished, FED:-1:recession, MM:-1:sell, COIN:-1:short, SPY:-1:sell, DELTA:-1:bad, FAST:-1:bad, INFO:-1:bad
['SPY: -0.2:bearish, CPI, inflation, Fed, FOMC, AAPL: -0.2:bearish, puts, calls, NVDA: 0.2:bullish, calls, LULU: -0.2:bearish, puts, BUF
['AAPL:0.8:Bullish, NVDA:0.8:Bullish, SPY:0.2:Neutral, AMC: -0.8:Bearish, F: -0.8:Bearish, NFLX: 0.8:Bullish, BAC: -0.8:Bearish']
['NVDA:-0.2:technical, FED:-0.8:judgement, AAPL:-0.6:products, SPY:-0.4:flat, RH:0.2:features, BABA:0.2:calls, CPI:-0.4:release, GOLD:0.
['No unique exchange listed stock tickers mentioned in the comments.']
['No tickers mentioned in the comments.']
['NVDA:0.5:positive, UXVY:-1.0:negative']
['NVDA:0.7:bulls, KMX:-0.3:puts, AAPL:-0.2:calls, TSLA:-0.7:bagholders, SPY:0.2:calls, NDVA:-0.3:puts, CPI:-0.2:leaked, TAI:-0.2:drills,
['NVDA:-0.5:irrational, pump, overbloaded, bearish, WD-40:0.5:blowout, earnings, saved, bullish, XOM:-0.5:oil, calls, bearish, MSFT:-0.5:panic, s
['NVDA:1:AI, force, algos, buy, SPY:0.5:green, close, Tesla:0.5:calls, QQQ:-0.5:fail, buying, TSLA:-0.5:crater, ABNB:0.5:puts, gain, long, AMD:1:AI, c
['NVDA:0.9:love, SPY:0.2:positive, QQQ:0.2:positive, TSLA:-0.3:negative, AI:-0.2:negative, AAPL:0.2:positive, FXHedge:0.0:neutral, MU:-0
['NVDA:0.8:bull, SPY:0.2:bear, GOOG:0.8:bull, AAPL:0.2:bear, GM:0.2:bear, AMC:0.2:bear, BBBY:0.2:bear, PLTR:0.2:bear, NIO:0.2:bear, TRLY:0.2:bear
['TSLA:-0.5:bearish, DIP:-1.0:bearish, SPY:-0.2:bearish, NFLX:-0.7:bearish, PLTR:-0.3:bearish, BBBY:-0.2:bearish, LEVI:-0.7:bearish']
['TSLA:1:IRA, NVDA:-0.5:dip, NFLX:-0.5:fuckery, SPY:-0.5:melt, AAPL:1:omega, TQQQ:1:win, GOOG:1:ripping, GME:-0.5:halted, COST:-0.5:wors
['DTE: -1.0:scam, NVDA: 0.0:neutral, AMD: 0.0:neutral, TSLA: 0.0:neutral, SPY: 0.0:neutral, QQQ: 0.0:neutral, CLX: 0.0:neutral, DRILL: 0
['AAPL:0.9:pumped, TSLA:0.2:calls, SPY:0.7:rally, NVDA:0.8:lesson, PLTR:0.2:clots, CPI:-0.7:softer, 420:0.7:ascended, MODS:0.5:thread, Jpow:0.2:
['NVDA:0.2:buy, puts, SPY:-0.2:sell, calls, GOOGL:0.2:buy, calls, TD:-0.2:short, ']
['APPL:-0.2:bearish, NFLX:0.2:decent, NVDA:-0.2:bearish, AMC:-1.0:angry, C3:-0.2:bearish, TSLA:-0.2:bearish, SPY:0.2:stable, MCD:-0.2:bearish']

<https://x.com/parmarsuraj99/status/1659434596440719361?s=20>

ChatGPT x Finance

 Web Browsing

PA

AMD230616C00160000



 Finished browsing

The AMD June 16, 2023 call option with a strike price of \$160.00 (AMD230616C00160000) has the following details:

- Current Price: \$0.54
- Previous Close: \$0.29
- Open: \$0.33
- Bid: \$0.53
- Ask: \$0.54
- Day's Range: \$0.29 - \$0.54
- Volume: 289 contracts
- Open Interest: 2.47k contracts¹.

Can LLM Demystify TC?

```
1 # Generate text
2 input_text = "Numerai True contribution (TC) is"
3 generated_text = model_pipeline(input_text, max_length=300, num_return_sequences=1, temperature=0.1, do_sample=True)
4 print(generated_text[0]["generated_text"])
```

Setting `pad\_token\_id` to `eos\_token\_id`:2 for open-end generation.

Numerai True contribution (TC) is the difference between the predicted returns and the actual returns. The TC is calculated by subtracting the actual from the predicted returns. The TC is calculated as follows:

$$TC = (\text{predicted_returns} - \text{actual_returns}) \times 1000$$

where predicted_returns is the predicted return of the model, and actual_returns is the actual return of the fund.

The TC is calculated for each round and is used to determine the payout for the round. The payout is calculated as follows:

$$\text{payout} = TC \times 0.01$$

where 0.01 is the payout factor.

The payout factor is a constant that determines how much the model is paid for each round. A payout factor of 0.01 means that the model is paid 1% of the TC.

The payout is calculated for each model and is used to determine the payout for the round. The payout is calculated as follows:

$$\text{payout} = TC \times \text{payout_factor}$$

where payout_factor is the payout factor for the round.

QLoRA Fine-tuned Mistral-7B on Numerai text (RC, Forum, Docs)

Forum: <https://forum.numer.ai/t/using-llms-to-create-trading-signals/6374/5?u=surajp>

Colab: [llm_meets_numerai_tc.ipynb](#)

Summary

To summarize,

1. Encoder models (BERT): **text -> contextual vectors**

1. Classify text (i.e., news, reports)
2. Cluster text files
3. Indexing and retrieval of text

2. Decoder models (GPT): **text -> generate text**

1. Generate new text
2. Can have conversations
3. Summarization
4. Think of it as an interpolator

Links

- <http://karpathy.github.io/2015/05/21/rnn-effectiveness/>
- <https://www.ruder.io/>
- <http://jalammar.github.io/>
- <https://github.com/karpathy/char-rnn>
- Neural Networks: Zero to Hero
- <https://huyenchip.com/blog/>
- <https://lilianweng.github.io/>
- <https://www.fast.ai/>

What's Next

- Download the slides
- Fine-tune a model on Colab Pro
- Take pre-trained finance encoder model to get embeddings (vectors) for news or make sentiment predictions
- Combine that with technical features
- Train a model for Signals that combines both Technical and News data for stocks
- Play with model and tell me your reviews:
 - [surajp/Mistral-7B-Instruct-v01-qlora-numerai-1epoch](#)

Thank You

Twitter: [@pamarsuraj99](#)

Discord: [@surajp1729](#)

Slides: <https://tinyurl.com/lm-toronto>

Code: within slides